

Элементы матричной алгебры. Обратные матрицы и определитель



Преподаватель



Ваше фото

ИМЯ ФАМИЛИЯ

Текущая должность

Количество лет опыта

Какой у Вас опыт - ключевые кейсы

Самые яркие проекты

Дополнительная информация по вашему усмотрению

Корпоративный e-mail

Социальные сети (по желанию)



ВАЖНО!

- Камера должна быть включена на протяжении всего занятия.
- Если у Вас возник вопрос в процессе занятия, пожалуйста, поднимите руку и дождитесь, пока преподаватель закончит мысль и спросит Вас, также можно задать вопрос в чате или когда преподаватель скажет, что начался блок вопросов.
- Организационные вопросы по обучению решаются с кураторами, а не на тематических занятиях.
- Вести себя уважительно и этично по отношению к остальным участникам занятия.
- Во время занятия будут интерактивные задания, будьте готовы включить микрофон или демонстрацию экрана по просьбе преподавателя.

ПЛАН ЗАНЯТИЯ

Введение

Основной блок

Задание для закрепления

Вопросы по основному блоку

Заключение



Стать IT-шником может каждый



ПОВТОРЕНИЕ ИЗУЧЕННОГО





Повторение

- Единичная матрица, операции над матрицами
- Тензоры
- Broadcasting
- Добавление размерностей



Стать IT-шником может каждый



ВОПРОСЫ





Введение

Векторы и операции над ними в практических задачах

Матрицы в практических задачах

Системы уравнений и обратная матрица

Определитель



Стать IT-шником может каждый



ОСНОВНОЙ БЛОК



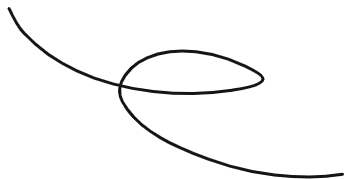
Векторы и операции над ними в практических задачах





Векторы —

это одномерные массивы. Возникают в практических задачах, когда мы имеем дело с упорядоченным набором чисел.



Векторы в практических задачах

```
# Вектор ежедневных расходов (в евро)
daily_expenses = np.array([10, 20, 15, 10, 30, 50, 50])

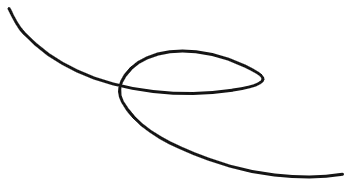
# Температура воздуха по месяцам (в градусах Цельсия)
temperature = np.array([1, 2, 6, 11, 16, 19, 21, 21, 16, 11, 6, 2])
```

Полезно знать



Операции над векторами —

это операции преобразования компонентов этих векторов.



Операции над векторами

```
# Перевод ежедневных расходов в доллары  
print(daily_expenses * 1.1)  
# [11. 22. 16.5 11. 33. 55. 55.]
```

```
# Перевод температуры в Фаренгейты  
print(1.8 * temperature + 32)  
# [33.8 35.6 42.8 51.8 60.8 66.2 69.8 69.8 60.8 51.8 42.8 35.6]
```


Стать IT-шником может каждый



ВОПРОСЫ



Стать IT-шником может каждый



ЗАДАНИЕ ДЛЯ ЗАКРЕПЛЕНИЯ





Подумайте:

Поясните, что вычисляется в следующих примерах:

1. `print(daily_expenses.sum())`
2. `print(temperature.mean())`
3. `print(daily_expenses.max())`
4. `print(temperature[5:8])`

Операции над векторами в практических задачах

```
# Доходы мужа по месяцам (в евро)
income_m = np.array([2000, 3000, 3500, 3000, 3000, 2500, 3500, 2000, 2500, 3000, 3500, 4000])

# Доходы жены по месяцам (в евро)
income_f = np.array([1500, 2500, 6000, 1100, 1600, 1900, 2100, 4200, 6000, 1100, 6000, 2000])

# Курс евро к доллару по месяцам
eur2usd = np.array([1.0779, 1.0716, 1.071, 1.096, 1.0879, 1.0834, 1.1056, 1.0916, 1.0686, 1.0561, 1.0804, 1.0902])

# Семейный доход в евро
income = income_m + income_f
print(income)
# [3500 5500 9500 4100 4600 4400 5600 6200 8500 4100 9500 6000]

# Семейный доход в долларах
income_usd = income * eur2usd
print(income_usd)
# [3772.65 5893.8 10174.5 4493.6 5004.34 4766.96 6191.36 6767.92
# 9083.1 4330.01 10263.8 6541.2]
```

Стать IT-шником может каждый



ВОПРОСЫ



Стать IT-шником может каждый



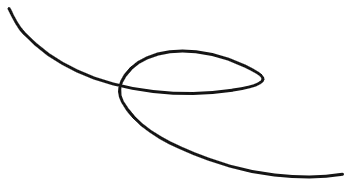
Матрицы в практических задачах





Матрицы —

это двумерные массивы. Возникают в практических задачах, когда нам нужно работать с несколькими родственными упорядоченными наборами чисел.



Матрицы в практических задачах

```
# Доходы членов семьи по месяцам
income_matrix = np.array([[2000, 3000, 3500, 3000, 3000, 2500, 3500, 2000, 2500, 3000,
3500, 4000], [1500, 2500, 6000, 1100, 1600, 1900, 2100, 4200, 6000, 1100, 6000, 2000]])

print(income_matrix)

# [[2000 3000 3500 3000 3000 2500 3500 2000 2500 3000 3500 4000]
#  [1500 2500 6000 1100 1600 1900 2100 4200 6000 1100 6000 2000]]
```

Пример

Кондитерская производит несколько видов десертов.

Для каждого необходимы мука, яйца и сахар, но в разном количестве.

Необходимое количество описать матрицей, в которой каждая строка описывает одну партию какого-то десерта, при этом первый столбец показывает количество муки в килограммах, второй столбец – количество яиц в штуках, третий – количество сахара в килограммах.

Пример

```
# Расход продуктов
goods = np.array([[0.5, 6, 0.1],
                  [1, 10, 0.5],
                  [0.3, 10, 0.6],
                  [0.5, 7, 0.3]])

print(goods)
# [[ 0.5  6.  0.1]
#  [ 1. 10.  0.5]
#  [ 0.3 10.  0.6]
#  [ 0.5  7.  0.3]]
```

Стать IT-шником может каждый



ЗАДАНИЕ ДЛЯ ЗАКРЕПЛЕНИЯ





Подумайте:

Пусть у нас также известна стоимость килограмма муки, одного яйца и килограмма сахара. Объясните, что происходит в этом фрагменте кода:

```
prices = np.array([0.75, 0.22, 0.88])

print(goods * prices)

# [[0.375 1.32  0.088]
#  [0.75  2.2   0.44 ]
#  [0.225 2.2   0.528]
#  [0.375 1.54  0.264]]
```


Нахождение суммарных затрат

$$\begin{bmatrix} 0.5 & 6 & 0.1 \\ 1 & 10 & 0.5 \\ 0.3 & 10 & 0.6 \\ 0.5 & 7 & 0.3 \end{bmatrix} * \begin{bmatrix} 0.75 \\ 0.22 \\ 0.88 \end{bmatrix} =$$

$$= \begin{bmatrix} (0.5 \times 0.75) + (6 \times 0.22) + (0.1 \times 0.88) \\ (1 \times 0.75) + (10 \times 0.22) + (0.5 \times 0.88) \\ (0.3 \times 0.75) + (10 \times 0.22) + (0.6 \times 0.88) \\ (0.5 \times 0.75) + (7 \times 0.22) + (0.3 \times 0.88) \end{bmatrix} = \begin{bmatrix} 1.783 \\ 3.39 \\ 2.953 \\ 2.179 \end{bmatrix}$$

Нахождение суммарных затрат (Python)

```
prices_col = prices.reshape(3,1)
expenses = goods @ prices_col
print(expenses)
print(expenses.sum())
```

```
# [[1.783]
# [3.39 ]
# [2.953]
# [2.179]]
# 10.305000000000001
```

Стать IT-шником может каждый



ВОПРОСЫ



Стать IT-шником может каждый



ЗАДАНИЕ ДЛЯ ЗАКРЕПЛЕНИЯ





Подумайте:

Фирма закупает ноутбуки и стационарные компьютеры для оборудования офисов на двух этажах здания.

- На первом этаже планируется поставить 20 стационарных компьютеров и 20 ноутбуков.
- На втором этаже – 12 стационарных компьютеров и 35 ноутбуков.

Компьютер стоит 850 евро, ноутбук – 1125 евро.

1. Запишите матрицу A , описывающую количество компьютеров и ноутбуков на каждом этаже.
2. Запишите вектор-столбец с ценами компьютеров и ноутбуков.
3. С помощью матричных операций вычислите стоимость оборудования на каждом этаже.



Решение:

```
# Количество оборудования
equipment = np.array([[20, 20], [12, 35]])
# Цены
prices = np.array([[850], [1125]])

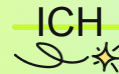
print(equipment @ prices)

# [[39500]
#  [49575]]
```


Системы уравнений и обратная матрица

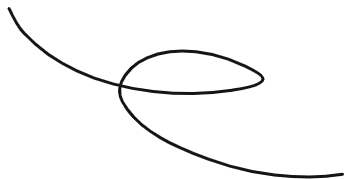


Полезно знать



Система уравнений —

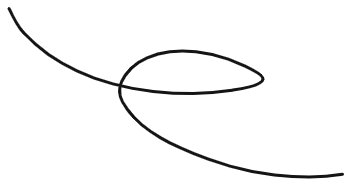
это набор уравнений с несколькими переменными.





Решение системы уравнений —

это набор значений переменных, при подстановке которых в уравнения все они обращаются в верные равенства.



Пример

Фирма закупает ноутбуки и стационарные компьютеры для оборудования офисов на двух этажах здания.

- На первом этаже планируется поставить 20 стационарных компьютеров и 20 ноутбуков.
- На втором этаже – 12 стационарных компьютеров и 35 ноутбуков.

Стоимость оборудования первого этажа должна составить 40000 евро, а второго этажа – 50000.

Обозначим через X - стоимость одного ПК, y - стоимость одного ноутбука.

Пример

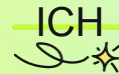
$$\begin{cases} 20x + 20y = 40000 \\ 12x + 35y = 50000 \end{cases}$$

$$A = \begin{bmatrix} 20 & 20 \\ 12 & 35 \end{bmatrix}, \quad X = \begin{bmatrix} x \\ y \end{bmatrix}, \quad B = \begin{bmatrix} 40000 \\ 50000 \end{bmatrix}$$

$$AX = B$$

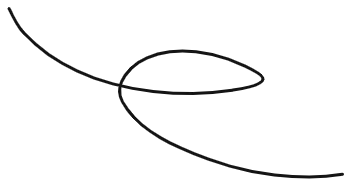
$$X = \frac{1}{A} B = A^{-1} B$$

Полезно знать



Обратные числа —

это числа, произведение которых равно единице.

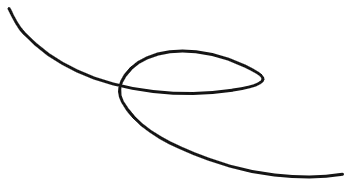


Полезно знать

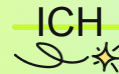


Взаимно обратные матрицы —

это матрицы, произведение которых равно единичной матрице.

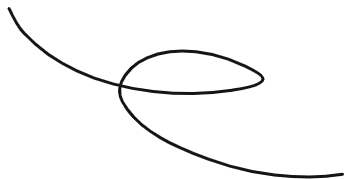


Полезно знать



`linalg.inv(matrix)` —

это метод в `numpy` для нахождения обратной матрицы к квадратной.



Метод `linalg.inv(matrix)`

```
equipment = np.array([[20, 20], [12, 35]])  
  
inverse = np.linalg.inv(equipment) # обратная матрица  
  
print(inverse)  
print(inverse @ equipment)  
print(equipment @ inverse)
```

Метод `linalg.inv(matrix)`

```
equipment = np.array([[20, 20], [12, 35]])  
expenses = np.array([[40000], [50000]])  
inverse = np.linalg.inv(equipment)  
print(inverse @ expenses)  
# [[ 869.56521739]  
# [1130.43478261]]
```

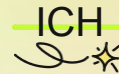
Стать IT-шником может каждый



ВОПРОСЫ



Стать IT-шником может каждый



ЗАДАНИЕ ДЛЯ ЗАКРЕПЛЕНИЯ





Подумайте:

Выполните проверку, что полученный вектор цен удовлетворяет матричному уравнению – подставьте значения в уравнения или выполните умножение матрицы на этот столбец.





Решение:

Выполните проверку, что полученный вектор цен удовлетворяет матричному уравнению – подставьте значения в уравнения или выполните умножение матрицы на этот столбец.

```
x = np.array([[ 869.56521739], [1130.43478261]])  
print(equipment @ x)
```

Стать IT-шником может каждый



Определитель



Пример

Фирма закупает ноутбуки и стационарные компьютеры для оборудования офисов на двух этажах здания.

- На первом этаже планируется поставить 20 стационарных компьютеров и 20 ноутбуков.
- На втором этаже – 30 стационарных компьютеров и 30 ноутбуков.

Стоимость оборудования первого этажа должна составить 40000 евро, а второго этажа – 50000.

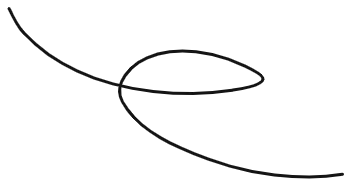
Нахождение обратной матрицы

```
equipment = np.array([[20, 20], [30, 30]])  
inverse = np.linalg.inv(equipment)  
print(inverse)  
  
# numpy.linalg.LinAlgError: Singular matrix
```



Singular matrix —

это ошибка в `numpy`, которая появляется, если найти обратную матрицу невозможно - если исходная матрица была квадратной.

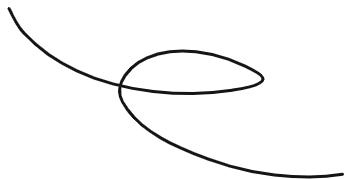




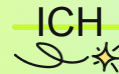
Определитель —

это специальная числовая характеристика матрицы.

Система уравнений с данной матрицей имеет единственное решение, только если ее определитель не равен нулю.

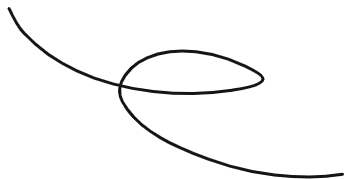


Полезно знать



`linalg.det(matrix)` —

это метод в `numpy` для нахождения определителя.



Нахождение определителя

```
equipment = np.array([[20, 20], [30, 30]])  
det = np.linalg.det(equipment)  
  
print(det)
```

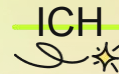
Стать IT-шником может каждый



ВОПРОСЫ



Стать IT-шником может каждый



ЗАДАНИЕ ДЛЯ ЗАКРЕПЛЕНИЯ





Подумайте:

Вычислите определитель предыдущей версии матрицы:

```
equipment = np.array([[20, 20], [12, 35]])
```


Стать IT-шником может каждый



ПРАКТИЧЕСКАЯ РАБОТА



Практическое задание

Матрица `goods`, как в примере выше, описывает количество муки, яиц и сахара в трех видах десертов некоторой кондитерской. Известно, что затраты на выпечку первого десерта составляют 2 евро, второго – 4 евро, третьего – 3 евро.

Найдите, сколько должны стоить мука, яйца и сахар для такой себестоимости десертов.

Для решения задачи составить систему уравнений, найти определитель этой системы. Если определитель не равен нулю, найти обратную матрицу и значения переменных.

Расход продуктов

```
goods = np.array([[0.5, 6, 0.1],  
                  [1, 10, 0.5],  
                  [0.3, 10, 0.6]])
```

Стать IT-шником может каждый



ОСТАВШИЕСЯ ВОПРОСЫ



Полезные ссылки

- [Действия с матрицами](#)
- [Linear algebra \(numpy.linalg\)](#)

Стать IT-шником может каждый



ЗАКЛЮЧЕНИЕ

